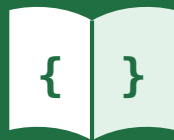

Capstone — Bookstore Platform

Exercises

Spring Boot Edu



2026-09-25

Contents

How to Work	2
Exercise 1 — Idempotency Key for Orders (Medium)	2
Exercise 2 — Cancel an Order (Medium)	3
Exercise 3 — Survive a Catalog Restart (Easy)	3
Extra Challenge (Optional)	4

How to Work

1. The starter code is in `capstone/exercise/order-service/`: a copy of the order service with the database migration, the entity fields and the endpoints already prepared. Find the `TODO` comments.
2. The tests start PostgreSQL and Kafka with Testcontainers and a fake catalog in-process (Docker needed, no other service):

```
./mvnw -Pexercises -pl capstone/exercise/order-service -am test
```

3. The exercise is done when all tests are green — the tests of the order service must stay green too.
4. If you get stuck, read the hints first, then the solution in `capstone/solution/order-service/`.

All three exercises are about the same question: what happens when a request, a message or a service is **repeated or missing**?

Exercise 1 — Idempotency Key for Orders (Medium)

Goal: a client whose `POST /api/orders` timed out sends it again. It must not order twice. The client sends an `Idempotency-Key` header; the same key of the same customer gives back the first order.

Tasks:

- `TODO` Exercise 1 (`order.OrderRepository`) — a derived query that finds an order by customer ID and idempotency key.
- `TODO` Exercise 1 (`order.OrderService.findByKey`) — no key: no earlier order; with a key: look it up.

Given: the migration `V3__status_and_idempotency.sql` (column and a unique partial index on (`customer_id`, `idempotency_key`)), the field in `Order`, and the controller: a found order is answered with 200 instead of 201.

Hints:

- The key belongs to a customer: two customers may use the same key.
- Why does `place()` also catch `DataIntegrityViolationException`? Think of two requests with the same key at the same millisecond — the unique index lets only one win.

Acceptance criteria: all 3 tests in `Exercise1IdempotencyTest` pass.

Estimated time: 30 minutes

Exercise 2 — Cancel an Order (Medium)

Goal: DELETE /api/orders/{id} cancels an order of the signed-in customer. The catalog gets the stock back, the order is marked CANCELLED, and an OrderCancelled event is published through the outbox.

Tasks (order.OrderService.cancel):

- TODO Exercise 2 — an unknown order or the order of someone else → OrderNotFoundException (404); an already cancelled order → OrderAlreadyCancelledException (409).
- Give the stock back with stock.release(orderId).
- In one transaction: order.cancel() and an OrderCancelled event in the outbox (OrderCancelled.TOPIC, key = order ID).

Hints:

- Release first or save first? ReleaseStock is idempotent in the catalog: releasing twice changes nothing. Which order of the two steps makes a retried DELETE safe after a failure in the second step?
- OrderCancelled has its own topic. Read its Javadoc: why not bookstore.orders? (Look at how the search service reads that topic.)
- Load the order again inside the transaction; changes of a managed entity are saved at commit.

Acceptance criteria: all 4 tests in Exercise2CancellationTest pass.

Estimated time: 45 minutes

Exercise 3 — Survive a Catalog Restart (Easy)

Goal: when the catalog restarts, the order service gets UNAVAILABLE for a moment. The reservation is retried twice before the customer gets a 503. “Not enough stock” is never retried.

Tasks:

- TODO Exercise 3 (stock.ResilienceConfiguration) — switch on Spring’s resilience annotations.
- TODO Exercise 3 (stock.StockClient.reserve) — retry on CatalogUnavailableException only: 1 call + 2 retries, with a short, growing delay.

Hints:

- Module 04, lesson 3.6: @EnableResilientMethods and @Retryable (Spring Framework 7, no extra library).

- Why is retrying reserve safe, but retrying a payment without a key would not be? (The catalog stores the reservation under the order ID.)
- The test expects exactly 3 attempts for a catalog that stays down.

Acceptance criteria: all 3 tests in Exercise3RetryTest pass.

Estimated time: 20 minutes

Extra Challenge (Optional)

1. **The trace through the outbox.** OrderPlaced starts a new trace (guide section 3.8). Store the W3C traceparent of the current span in a new outbox column, and let the relay continue that trace when it sends the record. In Grafana, the order should then be one trace from the gateway to the notification listener.
2. **Cancellations in the search index.** Let the search service consume bookstore.order-cancellations and subtract the quantities from sold — idempotently, like recordSale. Move OrderCancelled into contracts first.